

VARIABLES AND CONSTANTS

Outline

- Understanding how computer memory works
- Working with Variables
 - ✓ Variable Names
- Numeric Variable Types
 - ✓ Variable declaration
 - ✓ Using Typedef
 - ✓ Initializing Numeric Variables
- Working with Constants
 - ✓ Literal Constants
 - ✓ Symbolic constants

How Computer Memory works

- ✓ A computer uses random-access memory (RAM) to store information while it's operating. RAM is located in integrated circuits, or chips, inside your computer.
- ✓ RAM is volatile, which means that it is erased and replaced with new information as often as needed.
- ✓ Being volatile also means that RAM "remembers" only while the computer is turned on and loses its information when you turn the computer off.
- ✓ The *byte* is the fundamental unit of computer data storage.

- ✓ The RAM in your computer is organized sequentially, one byte following another.
- ✓ Each byte of memory has a unique address by which it is identified--an address that also distinguishes it from all other bytes in memory.
- ✓ Addresses are assigned to memory locations in order, starting at zero and increasing to the system limit.
- ✓ For now, you don't need to worry about addresses; it's all handled automatically by the C compiler.
- ✓ What is your computer's RAM used for? It has several uses, but only data storage need concern you as a programmer.
- ✓ Data is kept in your computer's RAM while the program is running.

Computer Memory



Variables

- ✓ **Dfn:** *A variable is a named data storage location in your computer's memory.*
- ✓ *By using a variable's name in your program, you are, in effect, referring to the data stored there.*
- ✓ The word variable is used to indicate that the values/information stored in the memory location can keep changing (**varying**) from time to time.
- ✓ You must ask the compiler to reserve an area of memory for a value you intend to use.
- ✓ To use variables in your C programs, you must know how to create variable names. In C, variable names must adhere to the following rules:

- ✓ The name can contain letters, digits, and the underscore character (_).
- ✓ The first character of the name must be a letter. The underscore is also a legal first character, but its use is not recommended.
- ✓ Variable names are case sensitive (that is, upper- and lowercase letters). Thus, the names **count** and **Count** refer to two different variables.
- ✓ C keywords(reserved words) can't be used as variable names. A keyword is a word that is part of the C language. (Check out the list of **Reserved Words** in C language.
- ✓ For many compilers, a C variable name can be up to 31 characters long. (It can actually be longer than that, but the compiler looks at only the first 31 characters of the name.)
- ✓ With this flexibility, you can create variable names that reflect the data being stored. For example, a program that calculates loan payments could store the value of the prime interest rate in a variable named **interest_rate**.
- ✓ The variable name helps make its usage clear.
- ✓ Although it might take a little more time to type descriptive variable names, the improvements in program clarity make it worthwhile.

✓ **Please Note:**

1. DO use variable names that are descriptive.
2. DO adopt and stick with a style for naming your variables.
3. DON'T start your variable names with an underscore unnecessarily.
4. DON'T name your variables with all capital letters unnecessarily.

Numeric Variable Types

- ✓ C provides several different types of numeric variables.
- ✓ You need different types of variables because different numeric values have varying memory storage requirements and differ in the ease with which certain mathematical operations can be performed on them.
- ✓ Small integers (for example, 1, 199, and -8) require less memory to store, and your computer can perform mathematical operations (addition, multiplication, and so on) with such numbers very quickly.
- ✓ In contrast, large integers and floating-point values (123,000,000 or 0.000000871256, for example) require more storage space and more time for mathematical operations.
- ✓ By using the appropriate variable types, you ensure that your program runs as efficiently as possible.

- ✓ C's numeric variables fall into the following two main categories:
 1. Integer variables hold values that have no fractional part (that is, whole numbers only).
 - Integer variables come in two flavors:
 - i. signed integer variables can hold positive or negative values,
 - ii. Unsigned integer variables can hold only positive values (and 0).
 2. Floating-point variables hold values that have a fractional part (that is, real numbers).

Variable Declarations

- ✓ Before you can use a variable in a C program, it must be declared.
- ✓ A variable declaration tells the compiler the name and type of a variable and optionally initializes the variable to a specific value.
- ✓ If your program attempts to use a variable that hasn't been declared, the compiler generates an error message.
- ✓ A variable declaration has the following form:
 - ***typename varname;***
- ✓ *typename* specifies the variable type
- ✓ *varname* is the variable name, which must follow the rules mentioned earlier.
- ✓ You can declare multiple variables of the same type on one line by separating the variable names with commas:
- ✓ ***int count, number, start; /* three integer variables */***

The typedef Keyword

- ✓ The **typedef** keyword is used to create a new name for an existing data type.
- ✓ In effect, typedef creates a synonym.
- ✓ For example, the statement
- ✓ **typedef int soic;**
- ✓ The above statement creates soic as a synonym for int.
- ✓ You then can use **soic** to define variables of type int, as in this example:
- ✓ **soic** count;
- ✓ **Note** that typedef doesn't create a new data type; it only lets you use a different name for a predefined data type.

Initializing Numeric Variables

- ✓ When you declare a variable, you instruct the compiler to set aside storage space for the variable.
- ✓ However, the value stored in that space--the value of the variable--isn't defined. It might be zero, or it might be some random "garbage" value.
- ✓ Before using a variable, you should always initialize it to a known value.
- ✓ You can do this independently of the variable declaration by using an assignment statement, as in this example:
 - ✓ **int count;** /* Set aside storage space for count */
 - ✓ **count = 0;** /* Store 0 in count */
- Note that this statement uses the equal sign (=), which is C's assignment operator.
- You need to be aware that the equal sign in programming is not the same as the equal sign in algebra.

- ✓ the same as the equal sign in algebra. If you write $x = 12$ in an algebraic statement, you are stating a fact: "x equals 12."
- ✓ In C, however, it means something quite different: "Assign the value 12 to the variable named x."
- ✓ You can also initialize a variable when it's declared.
- ✓ To do so, follow the variable name in the declaration statement with an equal sign and the desired initial value:
 - ✓ **`int count = 0;`**
 - ✓ **`double percent = 0.01, taxrate = 28.5;`**

Constants

- ✓ Like a variable, a *constant is a data storage location used by your program. Unlike a variable, the value stored in a constant can't be changed during program execution.*
- ✓ C has two types of constants, each with its own specific uses.

1.Literal Constants

- ✓ *A literal constant is a value that is typed directly into the source code wherever it is needed. Here are two examples:*

- ✓ `int count = 20;`

- ✓ `float tax_rate = 0.28;`

- ✓ The 20 and the 0.28 are literal constants.
- ✓ Note that one of these constants contains a decimal point, whereas the other does not.
- ✓ The presence or absence of the decimal point distinguishes floating-point constants from integer constants.

2. Symbolic Constants

- ✓ *A symbolic constant is a constant that is represented by a name (symbol) in your program.*
- ✓ *Like a literal constant, a symbolic constant can't change. Whenever you need the constant's value in your program, you use its name as you would use a variable name.*
- ✓ *The actual value of the symbolic constant needs to be entered only once, when it is first defined.*
- ✓ *Symbolic constants have two significant advantages over literal constants, as the following example shows.*
- ✓ *Suppose that you're writing a program that performs a variety of geometrical calculations.*
- ✓ *The program frequently needs the value π (3.14159) for its calculations.*
- ✓ *For example, to calculate the circumference and area of a circle with a known radius, you could write*
- ✓ *circumference = 3.14159 * (2 * radius);*
- ✓ *area = 3.14159 * (radius)*(radius);*

2. Symbolic Constants

- ✓ *A symbolic constant is a constant that is represented by a name (symbol) in your program.*
- ✓ *Like a literal constant, a symbolic constant can't change. Whenever you need the constant's value in your program, you use its name as you would use a variable name.*
- ✓ *The actual value of the symbolic constant needs to be entered only once, when it is first defined.*
- ✓ *C has two methods for defining a symbolic constant:*
 - ✓ *the **#define** directive and the **const** keyword.*
 - ✓ *The #define directive is one of C's preprocessor directives.*
 - ✓ *The #define directive is used as follows:*
 - ✓ ***#define** **CONSTNAME** *literal**

- ✓ This creates a constant named *CONSTNAME* with the value of *literal*.
- ✓ *literal* represents a literal constant, as described earlier.
- ✓ *CONSTNAME* follows the same rules described earlier for variable names.
- ✓ By convention, the names of symbolic constants are uppercase.
- ✓ This makes them easy to distinguish from variable names, which by convention are lowercase.

✓ Example

✓ **#define PI 3.14159**

- ✓ Note that #define lines don't end with a semicolon (;).
- ✓ #defines can be placed anywhere in your source code, but they are in effect only for the portions of the source code that follow the #define directive.
- ✓ Most commonly, programmers group all #defines together, near the beginning of the file and before the start of main().

Using the const keyword

- ✓ The second way to define a symbolic constant is with the **const keyword**.
- ✓ `const` is a modifier that can be applied to any variable declaration.
- ✓ A variable declared to be `const` can't be modified during program execution--only initialized at the time of declaration.
- ✓ Here are some examples:
 - ✓ **`const int count = 100;`**
 - ✓ **`const float pi = 3.14159;`**
 - ✓ **`const long debt = 12000000, float tax_rate = 0.21;`**
- ✓ `const` affects all variables on the declaration line. In the last line, `debt` and `tax_rate` are symbolic constants.
- ✓ If your program tries to modify a `const` variable, the compiler generates an error message: